

Chapitre 5 : Efficacité et gestion des ressources

I Utilisation des ressources

En programmation, l'efficacité d'un algorithme se mesure principalement à travers deux ressources limitées :

1. **La mémoire** : nécessaire pour stocker (temporairement ou durablement) les valeurs créées par les programmes.
2. **Le temps d'exécution** : le nombre d'opérations élémentaires effectuées.

En programmation fonctionnelle, la création fréquente de valeurs intermédiaires peut consommer de la mémoire. OCaml gère cette allocation dynamique de manière automatique grâce à un **Garbage Collector** (comme en Java), qui libère de l'espace en recyclant les structures de données devenues inaccessibles pour le programme.

II Appels de fonctions et pile d'exécution

Chaque appel de fonction non terminé consomme un espace en mémoire appelé **cadre d'activation** (ou *stack frame*) inséré dans la **pile d'exécution**. Ce cadre stocke les arguments de la fonction et ses variables locales.

Si une fonction effectue un trop grand nombre d'appels récursifs imbriqués, la pile peut saturer, ce qui déclenche une erreur fatale d'exécution.

✗ Attention ✗ Une fonction récursive non optimisée appliquée à une grande structure de données provoquera une levée d'exception : `Stack_overflow` (dépassement de la pile d'exécution).

III Récurrence terminale

Pour éviter la saturation de la pile, il convient de concevoir des fonctions utilisant la récursion terminale (*tail recursion*).

Définition : Une fonction est dite **récursive terminale** si l'appel récursif est la **toute dernière opération** effectuée par la fonction. Le résultat renvoyé est directement celui de l'appel récursif, sans aucun calcul supplémentaire en attente.

Grâce à cette propriété, le compilateur OCaml réalise une optimisation : il réutilise le même cadre d'activation dans la pile au lieu d'en empiler un nouveau. L'espace mémoire requis dans la pile devient constant $\Theta(1)$, transformant l'exécution en une boucle impérative.

💡 Exemple : Comparons l'implémentation naïve et l'implémentation optimisée à l'aide d'un accumulateur de la fonction factorielle.

Version naïve (Non récursive terminale) :

```
1 let rec fact n =  
2   if n = 0 then 1  
3   else n * fact (n - 1); (* Une multiplication reste en attente *)
```

Version optimisée (Récursive terminale) :

```
1 let fact_tail n =  
2   let rec fact_aux acc n =  
3     if n = 0 then acc  
4     else fact_aux (acc * n) (n - 1) (* Appel récursif terminal *)  
5   in fact_aux 1 n;
```

IV Structures de données : listes et files d'attente

Le choix de l'implémentation et de l'accès aux structures de données impacte fortement la complexité algorithmique.

A Efficacité sur les listes

Comme étudié précédemment, une liste OCaml est une liste simplement chaînée immuable.

- L'ajout en tête via l'opérateur `::` (*Cons*) est en $\Theta(1)$.
- Le parcours, le calcul de la taille (`List.length`) ou l'ajout en fin (`@`) requièrent un temps linéaire $\Theta(N)$ où N est la longueur de la liste.

B Modélisation d'une file d'attente (FIFO)

Une file d'attente suit la règle du *First In, First Out* (Premier entré, premier sorti). Nous voulons implémenter deux opérations principales : `put` (enfiler un élément) et `get` (défiler un élément).

1 Approche naïve avec une liste simple

Si l'on utilise une unique liste pour représenter la file :

- Soit `put` se fait en tête ($\Theta(1)$), mais `get` doit récupérer le dernier élément au bout de la liste ($\Theta(N)$).
- Soit `put` se fait en fin de liste ($\Theta(N)$), et `get` se fait en tête ($\Theta(1)$).

Dans les deux cas, l'une des opérations fondamentales possède une complexité linéaire coûteuse.

2 Approche optimisée : la règle des deux listes

Pour obtenir des opérations efficaces, on modélise la file à l'aide d'un enregistrement (*record*) contenant deux listes distinctes :

1. `instack` : la liste d'entrée, où sont stockés les nouveaux éléments insérés.
2. `outstack` : la liste de sortie, d'où sont extraits les éléments à défiler.

```
1 type 'a fifo = {
2   mutable instack : 'a list;
3   mutable outstack : 'a list;
4 };
```

Méthode : Algorithme de gestion de la file

- **Insertion** (`put`) : On ajoute l'élément en tête de `instack`. Cette opération est toujours en $\Theta(1)$.
- **Extraction** (`get`) :
 - Si `outstack` n'est pas vide, on extrait son premier élément ($\Theta(1)$).
 - Si `outstack` est vide, on **renverse** entièrement la liste `instack` dans `outstack`, puis on vide `instack`. L'extraction s'effectue ensuite sur la tête de `outstack`.

💡 **Exemple** : Implémentation complète des opérations de la FIFO :

```
1 exception Queue_empty;
2
3 let empty () = { instack = []; outstack = [] };
4
5 let put x queue =
6   queue.instack <- x :: queue.instack;
```

```

7
8 let get queue =
9     match queue.outstack with
10    | h :: r -> queue.outstack <- r; h
11    | [] ->
12        match queue.instack with
13        | [] -> raise Queue_empty
14        | _ ->
15            let rev_in = List.rev queue.instack in
16            match rev_in with
17            | h :: r -> queue.instack <- []; queue.outstack <- r; h
18            | [] -> raise Queue_empty;

```

Remarque : Bien que le retournement de la liste `instack` prenne un temps linéaire $\Theta(N)$, cette opération ne se produit que lorsque `outstack` devient totalement vide. Réparti sur l'ensemble des utilisations de la file, le coût de chaque opération `get` est en réalité de $\Theta(1)$ en moyenne.

 **Application :** Soit la fonction récursive naïve calculant le n -ième terme de la suite de Fibonacci : `let rec fib n = if n < 2 then n else fib (n-1)+ fib (n-2).`

Proposer une version récursive terminale compila-optimisée à l'aide d'une fonction auxiliaire et de deux accumulateurs.

[illegible]

Contents

I	Utilisation des ressources	1
II	Appels de fonctions et pile d'exécution	1
III	Récurrence terminale	1
IV	Structures de données : listes et files d'attente	2
A	Efficacité sur les listes	2
B	Modélisation d'une file d'attente (FIFO)	2
1	Approche naïve avec une liste simple	2
2	Approche optimisée : la règle des deux listes	2